

Influence of High-Level Program Structures on Branch Prediction Accuracy

Afshin Ganjoo and Nian-Feng Tzeng
Center for Advanced Computer Studies
University of Louisiana at Lafayette
Lafayette, LA 70504-4330, USA

Abstract

Current techniques do not take advantage of information regarding high-level program structures to improve branch prediction accuracy. The existing testing tools are inadequate in discovering these relationships. A new testing tool is thus developed to generate test programs. It allows the high-level program characteristics of these test programs to be varied. The tests conducted uncover the reasons behind the success and failure of different branch predictors, with the GAP and the path-based predictors chosen as examples for illustration. They also suggest ways the compiler can assist in improving the prediction accuracy. A new prediction scheme that takes advantage of these findings is introduced.

1. Introduction

To achieve higher execution speeds, current architectures use pipelining techniques to execute instructions in parallel. A major obstacle in this endeavor has been branch instructions. Prediction strategies have been developed to overcome this obstacle with considerable success. However, cheaper hardware and an increase in instruction length are opening the door to new prediction strategies that were considered impractical a few years ago.

Prediction strategies have been tested against standard benchmarks. Testing against these programs has been instrumental in determining the best prediction scheme. While the results were sufficient when a single branch prediction scheme had to be selected, when two or more schemes are to be used together, this method no longer suffices. A better testing tool is needed, especially one that could identify the strength and weakness of each prediction scheme. This level of analysis is necessary now because several branch prediction schemes are being incorporated into single processor architecture.

This paper describes a testing tool specifically designed for analyzing branch prediction schemes. Unlike conventional testing tools, this tool tests the prediction schemes with programs that have been created with their high-level characteristics conforming to the requests of the user. By varying the high-level characteristics of the created programs, the strength and weakness of each prediction scheme can be identified, providing insights into the reasoning behind their success or failure. The results can be employed to select an appropriate set of static and dynamic prediction schemes so that each would complement the others.

2. Motivation

Most dynamic branch prediction schemes are tested on simulators that operate on traces from assembly-language codes. While this measures their prediction accuracy, it provides very little other information. One is left to speculate as to why a predictor worked well with one program and failed with another. The situation becomes even more perplexing if one predictor performs better than another with one program, but fails to improve on the same predictor with another program. Very little reasoning can be provided from the existing tools operating on the assembly-language code. This is because the connection to the high-level program structures that they were generated from, is lost in the compiler's translation. We must ascertain if prediction schemes are sensitive to high-level branching conditions or not. If they are, then the compiler can attach bits to each branch instruction so the predictors can distinguish its originating high-level program structure and use that information to improve their prediction accuracy. This knowledge is even more significant now that several predictors are used in combination, as some predictors may be better suited to handle branches emanating from certain high-level program structures.

Further, dynamic prediction schemes search for paths and patterns of execution. It is the high-level program structures that establish these paths and patterns of execution. To gain an insight into the reason for success

or failure of a prediction scheme, it is absolutely crucial to test it with the high-level program structures intact so we can associate its behavior with specific structures. The manipulation of these structures gives us the apparatus we need to understand the nature of paths and patterns, and each scheme's response to them.

3. Branch Generator Test Tool

The test tool consists of the Generator program and the Analyzer program. The purpose of the Generator program is to generate test programs. It allows the changing of variables that control the high-level characteristics of the test programs. Although not every program characteristic is built in, enough has been included to make the test programs realistic. The following are the variables included.

- 1) Branch control: A variable sets the number of branches that are in the test program. The number of branches can have a direct impact on the amount of capacity aliasing experienced. Another variable contains the ratio of non-branch instructions per branch instruction. This variable determines the address allocated to each branch.
- 2) Procedure generator: A variable establishes how many procedures are created. A second variable determines the probability of calling a procedure within a basic block. Procedure calls and returns are among the contributors to history aliasing.
- 3) Flow control: The generated program is divided into segments. A variable determines the probability that their sequential execution is violated. Executing segments out of order contributes to conflict aliasing.
- 4) Register control: These registers are those used for branching purposes. The number of registers used can be set. Two branches that use the same register for branching are dependent, as long as the value of the register is not modified randomly or statically between their executions. This means that the behavior of one branch is a clue to the behavior of the other. The distance between two branches is equivalent to the number of branches between them. The longer the distance between the two dependent branches, the longer the global or path history needed to capture their relationship. A higher number of registers means longer distance between dependent branches.

The percentage of each register modification method can be set as well. There are three methods for modifying registers: random (e.g., $A = \text{Random}(10)$), static (e.g., $A = 20$), and dynamic (e.g., $A = A + 2$). Dynamic register modification establishes dependence between the next branch that uses the register after the register is modified and the last branch that used the register before the register

modification. Static register modification establishes dependence between the next branch that uses the register after the register is modified and the last branch that was executed before the register modification. Random register modification breaks any dependence between the next branch that uses the register after the register is modified and all branches before the register modification.

The probability that a register is modified in a basic block can be set. A higher register modification probability creates a more complex dependency structure. An increase in the random register modification probability can decrease dependency between branches.

- 5) Depth of branching: Horizontal and vertical depths for branching can be set. Horizontal depth refers to the level of nesting of if-then-else or loop statements within an encompassing branch structure. Vertical depth refers to the number of sequential (not nested within each other) if-then-else or loop statements within an encompassing branch structure. Deeper depths result in a more complex program structure. In the following example, loop A has a horizontal depth of two due to loops B and C (also due to loops B and D), and a vertical depth of one due to loop B.

```

For A = 1 to 5 do
  For B = 1 to 5 do
    For C = 1 to 5 do
      Next C
    For D = 1 to 5 do
      Next D
    Next B
  Next A

```

Loop B has a horizontal depth of one due to loop C (also due to loop D), and a vertical depth of two due to loops C and D. Loops C and D have horizontal and vertical depths of zero, since they encompass no nesting of branches.

- 6) Loop control: The percentage of branches constituted by loops can be set. The rest will be allocated to if-then-else statements. Also, the maximum range of loops can be established. The number of repetitions for a loop can be controlled randomly (e.g., For A = 1 to $\text{Random}(20)$), statically (e.g., For A = 1 to 20), or based on another loop. The controlling loop can be the immediate containing loop or a more distant one. The probability of selecting one over the other can be set.

In the above example, loop A is the immediate containing loop for loop B and the distant containing loop for loop C. Loop A controls the repetition of loop C.

```

For A = 1 to 5 do
  For B = 1 to 5 do
    For C = 1 to A do
      Next C
    Next B
  Next A

```

- 7) If-then-else control: The maximum number of elses in the if-then-else statements can be set. The number of elses in each if-then-else statement will be selected randomly up to the maximum number of elses allowed. Of the percentage of branches allocated to if-then-else statements, the percentage of if-then statements (no elses) can also be set. Relevant registers will determine which else in each if-then-else statement will execute.
- 8) Program repetition: The number of times the test program is repeated can be selected. This simulates program segments that are repeated many times, such as procedures and library routines. If the branches within these codes remain in the prediction table before they are visited again, it has the effect of having been repeatedly executed. Increase in program repetition can dramatically negate the effects of compulsory aliasing.

Once the test program has been generated, it is executed. It leaves an execution trace of its branches, which consists of branch addresses and directions. Then the Analyzer program is executed. This program includes the simulation of predictors that are to be tested. It takes in the trace and produces a prediction accuracy figure for each predictor.

4. Influential non-loop program characteristics

Two branch prediction schemes have been selected for testing purposes. The GAp (Global Adaptive branch prediction using Per-address pattern history tables) two-level adaptive predictor bases its prediction on the pattern of previous branch execution results (equivalent to Gselect) [3, 7]. The path-based predictor bases its prediction on the path taken to reach each branch [5, 6]. Ten bits of history has been allocated for the GAp and path-based predictors. The Branch Generator Test Tool has been used to test the suggested schemes. This will allow us to analyze the effectiveness of the proposed schemes on various program characteristics, identifying their strengths and weaknesses. One hundred test programs were generated and averaged for each mark on the x-axis of the following figures. All predictors were run on each test program.

In the following tests, programs with only if-then-else statements have been created. The reason for not including loops in the test programs is that they create too much interference for the two prediction schemes tested

here, diluting the effects of other high-level program structures tested. The purpose for these tests will be to determine if, and how dramatically, high-level program structures influence the prediction accuracy.

Two sets of tests were conducted. The first set assumes unlimited hardware. This assumption eliminates capacity and conflict aliasing and therefore tests the two predictors against history and compulsory aliasing only. Practically, testing with unlimited hardware assumption is equivalent to testing with programs whose code is too small to overload the prediction table. Such programs cause minimal capacity and conflict aliasing.

To test the predictors in the presence of capacity and conflict aliasing, the second set of tests assumes limited hardware. The test programs were designed to cause heavy capacity and/or conflict aliasing. These tests are equivalent to testing with programs whose active code segments are so large that they overload the prediction table.

Of the high-level program characteristics that were tested in the absence of loops, five showed to have a significant impact on branch prediction accuracy. These were the number of repetitions, the register modification method, the register modification probability, the number of registers, and the number of elses in if-then-else statements.

4.1. The “number of repetitions” factor

Most program segments, such as library routines and procedures, are repeated many times during the execution of programs. Also, smaller parts of programs are placed within loops and therefore repeat many times. The number of times a segment of a program is repeated without its branches being ejected from the predictor table has a significant impact on prediction accuracy.

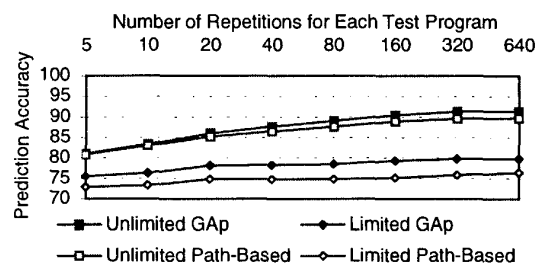


Figure 1: The influence of program repetition on prediction accuracy.

All cells are initialized as weakly taken. Therefore, low prediction accuracy is expected during the first few runs of the code just to overcome any compulsory aliasing. Further, not all paths and patterns for a branch will be encountered in the first few program repetitions. Since some program registers are modified randomly, the same

program will produce different execution results and therefore generate different paths and patterns for the same branch. That is why it is expected that the prediction accuracy continue to rise as the number of program repetitions increase. This rise in prediction accuracy is due to the new paths and patterns accessing previously not-visited cells and causing them to overcome their compulsory aliasing (Figure 1).

The rise in prediction accuracy is much slower in the presence of capacity and conflict aliasing. This is because the learning that is achieved is continuously undone due to destructive aliasing.

4.2. The “register modification method” factor

The method with which a register is modified plays an important role in establishing dependence between the branch that contains the register modification instruction and any branch that uses that register to determine its direction. The following example demonstrates the three methods:

```

10 If Y < 0 Then X = 10
20 Else If Y > 0 Then X = X + 1
30 Else If Y = 0 Then X = Random (10)
40 Else Error = True
100 If Even(X) Then Success = True

```

The register in segment 10 is modified statically. Both predictors can establish the dependence of the branch in segment 100 to the branch in this segment. The GAP predictor, however, cannot establish this dependence easily because it only records the direction of the previous branches, which is inadequate to identify the branch that was last executed.

The register is modified dynamically in segment 20. The same situation exists as before, except this time both predictors must go further back in the code to gather the necessary information about the register to be able to ascertain the dependence. The challenge is in reaching back far enough to discover critical information while keeping the path to it distinct.

The register is modified randomly in segment 30. Regardless of the prediction strategy employed to discover the dependence, it will be a failure. There is simply not enough information in branch addresses and directions to predict the behavior of the branch in segment 100 when its register is modified randomly. Note that randomly modified registers are not just registers that take a value from a random number generator. A register that gets its value from the user or from a database is also being modified randomly. The following is an example of such a scenario:

```

10 If NewInquiry = True Then Enter ModelYear
100 If ModelYear > 1990 Then InStock = True

```

Figure 2 identifies randomly modified registers as the most difficult to deal with for discovering dependencies. It also suggests that statically modified registers are the easiest to deal with, as expected. The greatest difference between prediction accuracy of different schemes occurs when branches are modified dynamically.

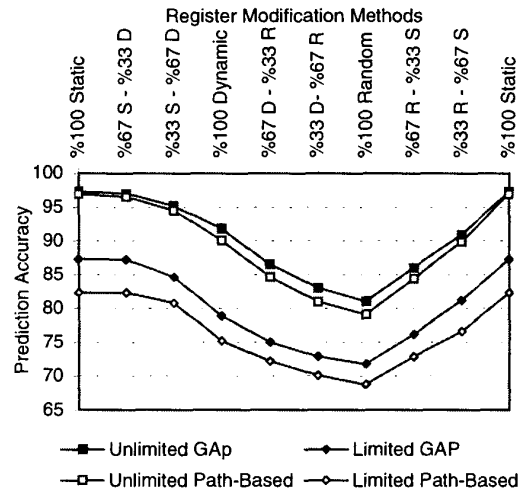


Figure 2: The influence of register modification methods on prediction accuracy.

4.3. The “register modification probability” factor

It has already been demonstrated that randomly modified registers sever dependencies. It is therefore expected that as the probability of register modification within basic blocks increase, the average distance between dependent branches decrease. This decrease, due to the higher probability of random register modification, reduces the advantage of deeper penetration into the code. The increase in register modification probability also results in more complex branch dependencies. Such increase in complexity will result in more paths and patterns being visited for each branch.

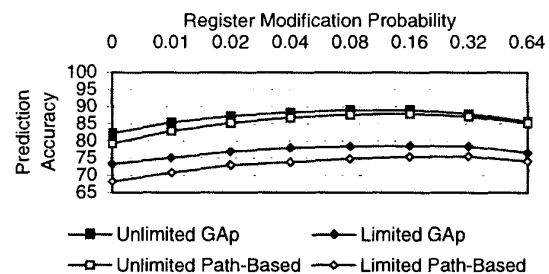


Figure 3: The influence of register modification probability on prediction accuracy.

Figure 3 displays the performance of the two predictors with limited and unlimited hardware. At register modification probability of zero, the registers maintain their initialization value and are never updated. At register modification probability of 0.32, any register can be modified in about a third of the basic blocks.

4.4. The “number of registers” factor

The significant role that the increase of the number of registers for branch usage plays is increasing the distance between dependent branches. Naturally, as this distance increases, more bits are needed to capture the dependencies. Figure 4 shows the ability of the predictors in dealing with distant dependencies.

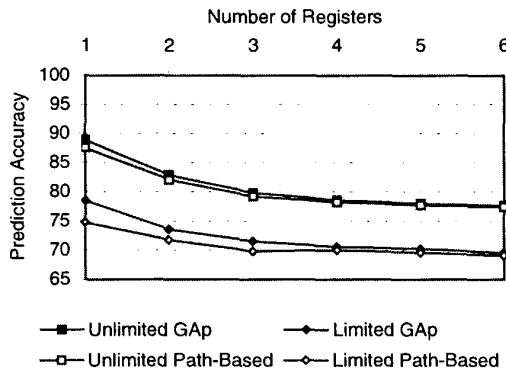


Figure 4: The influence of registers on prediction accuracy.

4.5. The “number of elses” factor

A moderate increase in the average number of elses can seriously reduce the prediction accuracy of many prediction schemes. There are two reasons for this. First, it increases the number of taken branches. This reduces the number of distinct branch patterns, which increases history aliasing. History aliasing occurs when long enough history is not maintained to properly distinguish the patterns leading to a branch. Therefore two different patterns leading to the same branch are treated the same. This is especially a problem for branches that follow if-then-else statements. Second, it puts more distance between dependent branches and therefore increases the need for more history bits. Both of these reasons negatively influence the GAp predictor, while only the latter affects the path-based predictor.

Figure 5 shows the effects of increasing the average number of elses on the GAp predictor. Its decline eventually levels off since as the average number of elses per if-then-else statement increase, more branches will be

taken, making them easier to predict. The path-based predictor has a more difficult time with fewer elses. This is because there are few distinct paths and they do not reveal much information about the registers.

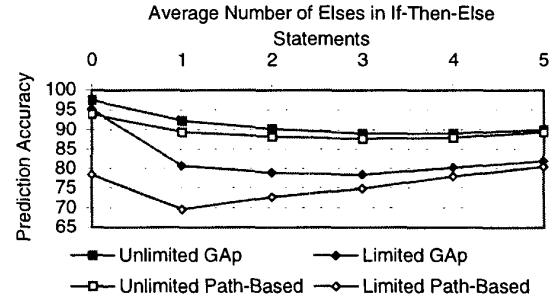


Figure 5: The influence of if-then-else complexity on prediction accuracy.

5. Influential loop-related program characteristics and the Loop-Free prediction scheme

Capacity and conflict aliasing are difficult problem to overcome, simply because they result from hardware limitation [4]. The obvious solution is to increase the number of lines in the predictor table, but that is not always practical. Another solution is to reduce the number of branches that need to be predicted dynamically. The approach suggested here is to use a static predictor for highly biased branches and a dynamic predictor for the rest. Since the static predictor requires minimal hardware, more hardware will be available for the dynamic predictor. By reducing the number of branches that must be dynamically predicted, all types of aliasing will be reduced.

Branch instructions can generally be classified as those resulting from loops (such as for-loops and while-loops) and those resulting from if-then-elses (such as if-then and case statements). Loop branches are highly biased. Furthermore, they typically loop more than there are history bits to capture their full behavior. Therefore, they are perfect candidates for static prediction. If-then-else branches, on the other hand, are usually not biased and can benefit from a dynamic predictor that learns paths and patterns.

The compiler is responsible for classifying branches as loop generated or if-then-else generated. It can store a bit with each branch instruction to identify the class it belongs to at compile time. If the compiler sets the bias of for-loops as not-taken and the bias of while-loops as taken, then an additional bit must be set by the compiler so that the processor can identify the bias of each loop subclass during execution.

The loop-free prediction scheme uses a static predictor for branches that resulted from loop structures and a dynamic predictor for the rest [1]. This will reduce the number of branches that must be predicted dynamically, which in turn reduces aliasing.

Figure 6 portrays the results of varying the percentage of loop statements. Clearly, as the percentage of loops increases (and the percentage of if-then-else statements decreases) prediction accuracy also increases. The results are consistent with the assumption that scientific programs (floating-point programs) are predicted more accurately because they have more loops.

The Branch Generator translates all loop branches to bias not-taken. Loop-free predictors, by using a static predictor to predict loops as not-taken, provided 5 to 8 percentage points higher accuracy.

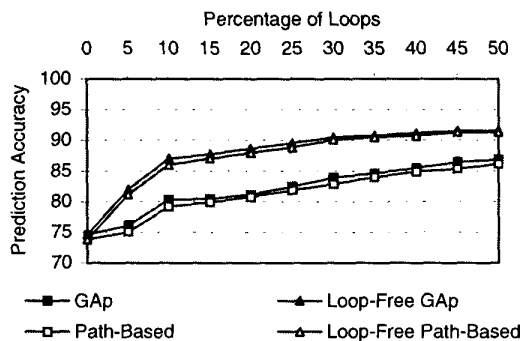


Figure 6: The influence of loop percentage on prediction accuracy.

Figure 7 exhibits the accuracy of the predictors when the range of the loops varies. It is evident that the higher the repetition of loops, the higher the accuracy. This is because the predictors can learn the paths and patterns of branches more thoroughly. Further, they get more chances to put that learning into use when loop repetitions are higher.

Since ten bits of history are maintained, when loops range from two to five, normal predictors should be able to predict their behavior and even improve on loop-free predictors. However, interference from if-then-else branches proved even more damaging. The difference between normal predictors and loop-free predictors is greater at low loop repetition than at high loop repetition.

Figure 8 shows the results of increasing the complexity of the test programs. As vertical or horizontal depths increase, so does the size of program segments stuck in loops. This in turn increases the prediction accuracy. The loop-free predictors exhibit a dramatic increase in accuracy for test programs that do not have deep structures. The difference in accuracy decreases as structural depth increases.

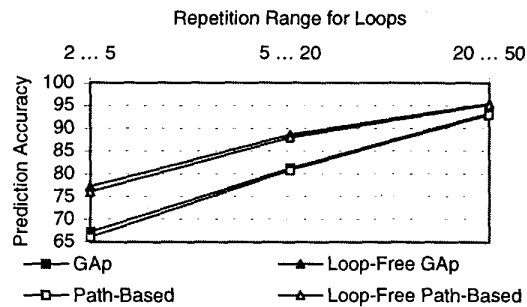


Figure 7: The influence of loop repetition range on prediction accuracy.

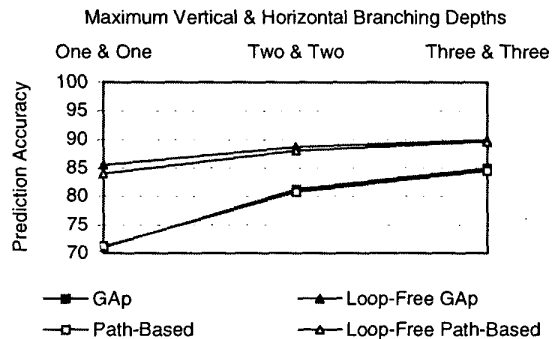


Figure 8: The influence of horizontal and vertical depths on prediction accuracy.

Figure 9 depicts the accuracy of the predictors as the size of test programs varies. In this test, each predictor table consists of 32 lines, selected by the branch addresses, and 1024 columns of 2-bit registers, identified by the history bits. Even though test programs with fewer branches cause less capacity aliasing, they also have fewer loops. This in turn decreases prediction accuracy. As the number of branches increase beyond a certain point, the destructive power of capacity aliasing eliminates the advantage of having more loops. The loop-free predictors maintained a healthy advantage over their normal counterparts.

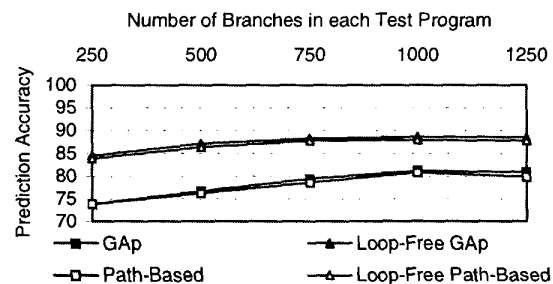


Figure 9: The influence of program size on prediction accuracy.

6. Conclusion

The research work outlined here shows that high-level program structures do influence the prediction accuracy of predictors. The test tool described uncovers this influence and opens the way for the compiler to provide hints to the hardware to improve prediction accuracy.

Five program characteristics were identified as having an impact on branch prediction accuracy of the GAP and path-based predictors. The predictors' response to the "program repetition" factor suggests that programs that make continuous use of functions and library routines can be predicted more accurately than those that do not repeat any of their segments. Results also suggest that those program segments that are repeated continuously should not be evicted from the predictor table. This requires the separation of the predictor table into two segments, one for heavily repeated program segments and one for program segments that do not repeat much. The compiler takes on the responsibility of identifying the branches in the repetitious program segments and relaying that information to the hardware.

The predictors' response to "register modification method" factor explains why predictors have a more difficult time accurately predicting the behavior of branches in database and user-interactive programs than in scientific programs. The higher the percentage of randomly modified registers, the lower the prediction accuracy. If the compiler can identify the branches that use randomly modified registers, it can set a flag for the hardware so it does not waste resources to predict these branches. Elimination of these branches from the predictor table reduces aliasing and should improve the overall prediction accuracy.

Testing against the "register modification probability" and "number of registers" factors showed the importance of history length. It suggests that a varying history length can work better than a constant history length [also see 2]. Based on the distance between dependent branches, the compiler can inform the hardware to use fewer or greater history bits to predict the branches more accurately without wasting resources.

The "number of elses" factor clearly shows the advantage of the GAP predictor over the path-based predictor with limited hardware. Again the compiler can use this information to relieve the predictor table from capacity and conflict aliasing.

The number and range of loops play an important role in determining the accuracy of predictors. Prediction accuracy increases as the number and range of loops increase. This is not only due to the relative ease of

predicting loop branches, but also due to the increase in repetition of other branches within loop statements.

By predicting loop branches according to their biases as identified by the compiler, the loop-free GAP and path-based predictors noticeably improve on normal GAP and path-based predictors. The lower the prediction accuracy, the greater the improvement. Because of capacity and conflict aliasing, loop branches are hashed with if-then-else branches. A loop branch will typically be translated by the compiler to bias not-taken. An if-then-else branch will be taken if its condition fails, which is more often the case. This difference in behavior creates destructive aliasing. Loop-free prediction scheme accepts an occasional misprediction from static prediction of loop branches, but eliminates all the mispredictions of loop branches caused by capacity and conflict aliasing. Since loops are heavily biased, the occasional misprediction of static prediction is not as damaging as the mispredictions caused by capacity and conflict aliasing.

These test results suggest that using the compiler to classify branches based on high-level program structures can increase the prediction accuracy of branch predictors. Even though a simple schemes have been offered here, a notable improvement has been achieved. More complex schemes can be devised by further classifying high-level program structures and incorporating them into the overall prediction strategy.

References

- [1] Afshin Ganjoo, "New Techniques for Minimizing Branch Penalties in Pipelined Processors," Dissertation, University of Louisiana at Lafayette, May 2000.
- [2] T. Juan, S. Sanjeevan, and J.J. Navarro, "Dynamic History-Length Fitting: A third level of adaptivity for branch prediction," *Proc. 25th International Symposium on Computer Architectures*, June 1998, pp. 155-166.
- [3] S. McFarling, "Combining Branch Predictors," DEC Western Research Laboratory, TN-36, June 1993.
- [4] P. Michaud, A. Seznec, and R. Uhlig, "Trading Conflict and Capacity Aliasing in Conditional Branch Predictors," *Proceedings 24th International Symposium on Computer Architectures*, May 1997, pp. 292-303.
- [5] R. Nair, "Dynamic Path-Based Branch Correlation," *Proceedings of the 28th International Symposium on Microarchitecture*, 1995, pp. 15-23.
- [6] S. Reches and S. Weiss, "Implementation and Analysis of Path History in Dynamic Branch Schemes," *IEEE Transactions on Computers*, Vol. 47, No. 8, August 1998, pp. 907-912.
- [7] T.-Y. Yeh and Y.N. Patt, "A Comparison of Dynamic Branch Predictors that Use Two Levels of Branch History," *Proceedings 20th International Symposium on Computer Architectures*, May 1993, pp. 257-266.